基于 MindSpore 的 RNN 情感分类实验（IMDB 影评）

项目	说明
适用课程	昇腾 AI 生态实验
适用对象	深圳大学本科生
实验类型	验证型（含对比调参）
难度	初级
硬件环境	昇腾服务器（Ascend 310P3），本实验以 **CPU 模式**运行
软件环境	MindSpore 2.9.0（服务器已安装）+ Python 3.9 + VS Code Remote-SSH
数据集	IMDB 影评：训练 2.5 万条、测试 2.5 万条，随本指导书提供 `aclImdb_v1.tar.gz`（81 MB）
词向量	GloVe 100 维预训练词向量，已裁剪成本实验词表可用的 3.9 万词，`glove_imdb_100d.txt`（32 MB）
建议学时	2 学时（训练 6~8 分钟）

> **配套文件**（与本文件同一目录）：`aclImdb_v1.tar.gz`（影评语料，不用解压，代码直接读 tar）、`glove_imdb_100d.txt`（预训练词向量）、`基于MindSpore的RNN情感分类实验.ipynb`（分格可运行的完整代码，逐格执行即可产出全部截图；下文各任务的代码与 notebook 一一对应）。

目录

1. 实验目的
2. 实验环境
3. 背景知识
    - 一、为什么不能把文本直接丢进全连接层
    - 二、词向量：从 one-hot 到 GloVe
    - 三、从 RNN 到 LSTM
    - 四、双向 LSTM 与“取最后隐状态”
4. 数据集说明
5. 实验总体设计
6. 实验步骤
    - 任务零：确认数据就位
    - 任务一：导入依赖与 CPU 初始化
    - 任务二：定义超参数
    - 任务三：读取影评并分词

    - 任务四：构建词表
    - 任务五：加载 GloVe 生成词向量矩阵
    - 任务六：搭数据流水线
    - 任务七：定义 RNN 模型
    - 任务八：训练
    - 任务九：在测试集上评估
    - 任务十：用模型给句子打分
7. 实验结果与思考题
8. 常见问题排查
9. 附录 A：完整源码（可直接运行）
10. 附录 B：耗时与调参记录

一、实验目的

1. 理解**情感分类**这个 NLP 经典任务：给一段影评，判断它是好评还是差评。
2. 掌握文本进神经网络的完整预处理链路：分词 → 建词表 → 查预训练词向量 → 统一长度 → 分批。
3. 理解**循环神经网络（RNN）**为什么适合处理序列，以及 LSTM 的门控机制解决了普通 RNN 的什么问题。
4. 用 MindSpore 亲手搭一个**双向 LSTM** 分类器，完成训练、评估、推理，并解释准确率是怎么来的。

二、实验环境

本实验在课程服务器上完成，本地电脑只需要 VS Code。环境要点：

- 操作系统：openEuler 22.03（aarch64）
- Python：3.9（`/usr/bin/python3`）
- 深度学习框架：MindSpore 2.9.0（服务器已安装，无需自己装）
- 开发方式：VS Code + Remote-SSH 远程连接服务器，代码直接写在服务器上
- 服务器不能访问外网，**数据集和词向量已随本指导书提供**，不需要也不要去 `wget`、`pip install`

> ⚠️ **注意（本实验最容易踩的坑）**：服务器装的是昇腾版 MindSpore，不显式指定 CPU 时它会去找昇腾芯片，直接报 `No module named 'tbe'` 然后退出。所以**任务一的第一个代码格里必须先执行下面这句**，后面所有代码才跑得起来：
>
> ```
> import mindspore as ms
> ms.set_context(mode=ms.PYNATIVE_MODE, device_target="CPU")
> ```

> ⚠️ **注意**：`import mindspore` 时会刷出几条 `Cannot find the tbe operator implementation` / `No module named 'te'` 的告警，这是昇腾套件没配环境变量导致的，指定 CPU 之后不影响任何计算，可以忽略。

- 连接服务器的方法见《学生连接排障手册》，本指导书假设你已经在 VS Code 里打开远程目录并运行 Python。

> ✅ **检查点**：在 VS Code 终端执行 `python3 -c "import mindspore; print(mindspore.__version__)"`，能打印 2.9.0 即环境正常。

截图 1

```
[stu1@localhost AddKernelInvocation]$ python3 -c "import mindspore; print(mindspore.__version__)"
/home/stu1/.local/lib/python3.9/site-packages/numpy/core/getlimits.py:549: UserWarning: The value of the smallest subnormal for <class 'numpy.float64'> type is zero.
  setattr(self, word, getattr(machar, word).flat[0])
/home/stu1/.local/lib/python3.9/site-packages/numpy/core/getlimits.py:89: UserWarning: The value of the smallest subnormal for <class 'numpy.float64'> type is zero.
  return self._float_to_str(self.smallest_subnormal)
/home/stu1/.local/lib/python3.9/site-packages/numpy/core/getlimits.py:549: UserWarning: The value of the smallest subnormal for <class 'numpy.float32'> type is zero.
  setattr(self, word, getattr(machar, word).flat[0])
/home/stu1/.local/lib/python3.9/site-packages/numpy/core/getlimits.py:89: UserWarning: The value of the smallest subnormal for <class 'numpy.float32'> type is zero.
  return self._float_to_str(self.smallest_subnormal)
2.9.0
```

三、背景知识

3.1 为什么不能把文本直接丢进全连接层

图片是固定的 28×28 个像素，位置固定、维度固定。文本不行：一句话长一点短一点都合法，而且**词的顺序决定意思**——“电影不好看”和“电影好看”差别就在一个“不”字。

如果把词袋（bag of words）直接喂给全连接层，模型只统计“出现了哪些词”，完全丢掉顺序，就区分不了这两句。要保留顺序，就得让网络**一个词一个词地读**，并且把“读到第几个词”这个状态带到下一步——这就是循环神经网络。

3.2 词向量：从 one-hot 到 GloVe

神经网络只认数字，所以每个词要先变成向量。

表示方式	长什么样	问题
one-hot	词表 3 万维，只有一位是 1	维度爆炸、任意两个词距离都一样，学不出“好”和“棒”是近义词
词向量（Word2Vec/GloVe）	100 维稠密向量	语义相近的词向量也相近，且参数少

GloVe 是把语料里“词与词的共现规律”压缩成向量的产物。用它的好处是：**你只有 2.5 万条影评，但词向量是从几十亿词的语料里学来的**，等于借了别人的经验，小数据集也能训出像样的模型。

3.3 从 RNN 到 LSTM

最朴素的 RNN 每一步只做一件事：把“上一步的隐状态”和“当前词向量”混在一起，算出新的隐状态：

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

它的问题是**梯度消失**：句子一长，前面的信息传到后面就只剩个影子，模型记不住“前面说过‘不’”。

LSTM 加了三个门（输入门、遗忘门、输出门）和一个细胞状态，本质是给记忆加了个“开关”：该记的记、该忘的忘。MindSpore 里就是一行 `nn.LSTM`，门内部怎么算不需要你手写，但你要知道**它能缓解长距离依赖**，这正是情感分类需要的（“不”可能出现在评价词的十个词之前）。

3.4 双向 LSTM 与“取最后隐状态”

单向 LSTM 读一句话是从左到右，读到最后一个词时，隐状态里压着整句的信息。但情感词的位置不固定，有时候关键词在句首，从左往右读到结尾反而把它“冲淡”了。

双向 LSTM 同时正着读一遍、反着读一遍，得到两个方向的最后隐状态：

- `h[-2]`：正向读完最后一个词的状态

- `h[-1]`：反向读完最后一个词（也就是原句第一个词）的状态

把这两个 64 维向量拼起来得到 128 维，再送进全连接层打一个分。这样句子开头和结尾的信息都被保留。

四、数据集说明

4.1 IMDB 影评数据集

IMDB 是情感分类最常用的公开数据集，2.5 万条训练 + 2.5 万条测试，正负各一半。本实验直接读 `aclImdb_v1.tar.gz`，**不需要解压**，目录结构如下：

```
aclImdb/
    train/
        pos/   12500 篇好评（*.txt，一篇一个文件）
        neg/   12500 篇差评
    test/
        pos/   12500 篇
        neg/   12500 篇
```

一条影评长这样（截断）：

```
Bromwell High is a cartoon comedy. It ran at the same time as some other
programs about school life, such as "Fast Times at Ridgemont High"...
```

标签很简单：`pos` 目录 → 1，`neg` 目录 → 0。

4.2 GloVe 词向量文件

原始 GloVe 6B 有 40 万词、822 MB（压缩包），其中大部分词跟影评无关。本实验提供的是**裁剪版** `glove_imdb_100d.txt`：只保留在 IMDB 语料里出现过 5 次以上的词，共 38,807 行、32 MB。每行格式是"词 + 100 个浮点数"：

```
the 0.418 0.24968 -0.41242 0.1217 0.34527 ...  （共 100 个数）
good 0.2029 -0.2465 0.2085 -0.2305 0.1369 ...
```

> 💡 **提示**：如果某个词在 GloVe 里没有（比如生僻词、拼写错误），代码会用一个随机向量占位；这种词在训练中会慢慢被调整成合适的向量。

4.3 本实验用的规模

服务器只有 CPU，为了让一次实验在 10 分钟内跑完，规模做了缩减（**这些都在任务二里写成参数，想调直接改**）：

项目	本实验取值	原始论文/官方 notebook
训练样本	25000 条	25000 条

项目	本实验取值	原始论文/官方 notebook
测试样本	4000 条	25000 条
词表	训练集里出现 ≥5 次的词，约 3.2 万	GloVe 全量 40 万
最大序列长度	200 个词	500
训练轮数	3	5

> 💡 **提示**：影评长度中位数约 170 个词，截断到 200 已经覆盖大部分内容；测试集只取 4000 条是为了让每轮评估快一点（全量 2.5 万条评估一次要多花 1 分钟）。

五、实验总体设计

整个实验是一条“文本 → 向量 → 序列建模 → 二分类”的链路：

1

读影评、分词

从 aclImdb tar 里逐篇读 txt，转小写、去标点，切成词序列（一篇平均 230 个词）

▼

2

建词表

统计词频，只保留出现 ≥5 次的词：约 **3.2 万个词**；另加 <pad> 和 <unk>

▼

3

查 GloVe 拿词向量

每个词取 100 维预训练向量，拼成 embedding 矩阵（3.2 万 × 100）

▼

4

统一长度、分批

Lookup 把词换成 id，PadEnd 截断/补齐到 200 步，每批 32 篇

▼

5

过 LSTM 抽时序特征

双向 LSTM 逐词读过去，取正反两个方向的最后隐状态，拼成 128 维

▼

6

全连接 + Sigmoid

128 维 → 1 个分数，Sigmoid 压到 0~1：**>0.5 判积极**，否则消极

▼



图 1　RNN 情感分类实验的七个环节

对应到实验步骤：

环节	对应任务	产出
读影评、分词	任务三	每篇影评一个词列表
建词表	任务四	约 3.2 万词的词表 + 特殊符号 `<pad>`、`<unk>`
查 GloVe 拿词向量	任务五	embedding 矩阵（词表大小 × 100）
统一长度、分批	任务六	每个 batch 形状 `(32, 200)` 的 id 张量
双向 LSTM 抽特征	任务七	每篇影评 128 维向量
全连接 + Sigmoid	任务七	0~1 之间的"好评概率"
训练与评估	任务八、九	准确率、模型文件

六、实验步骤

> 以下代码按顺序写进一个文件 `imdb_rnn.py`（或者 Jupyter Notebook 里一格一格跑）。**所有代码都放在与 `aclImdb_v1.tar.gz`、`glove_imdb_100d.txt` 同一个目录下运行**，代码里用的是相对路径。

任务零：确认数据就位

服务器上不需要解压语料，代码直接读 tar。只要确认两个文件在就行：

```
cd ~/gaoquan/1.基于MindSpore的RNN情感分类    # 换成你自己的实验目录
ls -lh aclImdb_v1.tar.gz glove_imdb_100d.txt
```

应该看到两个文件：约 81 MB 和 32 MB。

> 🔴 **截图 2**

```
[stu1@localhost 基于MindSpore的RNN情感分类实验]$ ls -lh
total 113M
-rw-rw-r--. 1 stu1 stu1 81M Oct 22 22:31 aclImdb_v1.tar.gz
-rw-rw-r--. 1 stu1 stu1 32M Oct 22 22:31 glove_imdb_100d.txt
-rw-rw-r--. 1 stu1 stu1 15K Oct 22 22:31 基于MindSpore的RNN情感分类实验.ipynb
[stu1@localhost 基于MindSpore的RNN情感分类实验]$
```

> ✅ **检查点**：两个文件都在同一个目录，且大小与上面一致（说明上传完整，没传断）。

任务一：导入依赖与 CPU 初始化

```
import collections
import os
import re
```

```
import string
import tarfile
import time

import numpy as np
import mindspore as ms
import mindspore.dataset as ds
import mindspore.nn as nn
import mindspore.ops as ops

# 本服务器是昇腾版 MindSpore，必须显式指定 CPU，否则去初始化昇腾芯片会报错
ms.set_context(mode=ms.PYNATIVE_MODE, device_target="CPU")
```

依赖都是服务器自带的（numpy、mindspore），不需要 pip install。

任务二：定义超参数

把所有可调的量集中放一处，改参数只改这里：

```
DATA_DIR = "."                        # 数据文件所在目录
TAR_PATH = os.path.join(DATA_DIR, "aclImdb_v1.tar.gz")
GLOVE_PATH = os.path.join(DATA_DIR, "glove_imdb_100d.txt")

MAX_LEN = 200         # 每篇影评截断/补齐到多少个词
BATCH_SIZE = 32       # 每批多少篇
EPOCHS = 3            # 训练轮数
LR = 1e-3             # 学习率
HIDDEN_SIZE = 64      # LSTM 每个方向的隐状态维度
EMBED_DIM = 100       # 词向量维度（GloVe 100d，不要改）
MIN_FREQ = 5          # 词表中保留的最低词频
TRAIN_PER_CLASS = 12500   # 训练集每类取多少条（12500 = 全量）
TEST_PER_CLASS = 2000     # 测试集每类取多少条
```

参数	作用	调大/调小的后果
MAX_LEN	每篇截断到多少词	调大保留更多信息但更慢（耗时线性增长）；调小丢信息
BATCH_SIZE	每批样本数	调大训练更稳但更吃内存
EPOCHS	训练轮数	轮数太少欠拟合；太多在 CPU 上耗时间
LR	学习率	太大会震荡不收敛，太小收敛极慢
HIDDEN_SIZE	LSTM 隐状态维度	调大表达力强但慢、易过拟合
TRAIN_PER_CLASS	训练集规模	调大更准但更慢（12500 是全量）

> 💡 **提示**：想快点看到结果，可以把 EPOCHS 改成 2、TRAIN_PER_CLASS 改成 5000，1~2 分钟就能跑完，准确率大概掉 5 个百分点。

任务三：读取影评并分词

写一个类，从 tar 里按目录逐篇读，转小写、去掉标点、切成词列表：

```
PAT = re.compile(r"aclImdb/(train|test)/(pos|neg)/.*\.txt$")
PUNCT = str.maketrans("", "", string.punctuation)


class IMDBData:
    """从 tar 包里按 split 读影评，每类最多取 per_class 条。"""

    def __init__(self, split, per_class, seed=42):
        docs, labels = [], []
        counts = {1.0: 0, 0.0: 0}
        with tarfile.open(TAR_PATH) as tarf:
            for member in tarf:
                m = PAT.match(member.name)
                if not m or m.group(1) != split:
                    continue
                label = 1.0 if m.group(2) == "pos" else 0.0
                if counts[label] >= per_class:
                    continue
                raw = tarf.extractfile(member).read().decode("utf-8", "ignore")
                # 换行换成空格、去掉标点、转小写，再按空格切词
                tokens = raw.replace("\n", " ").translate(PUNCT).lower().split()
                docs.append(tokens)
                labels.append(label)
                counts[label] += 1
                if counts[0.0] >= per_class and counts[1.0] >= per_class:
                    break
        # 打乱顺序（tar 里是“先把 pos 读完再读 neg”，不打乱会让训练/验证集类别失衡）
        order = np.random.default_rng(seed).permutation(len(docs))
        self.docs = [docs[i] for i in order]
        self.labels = [labels[i] for i in order]

    def __getitem__(self, idx):
        return self.docs[idx], self.labels[idx]

    def __len__(self):
        return len(self.docs)


t0 = time.time()
train_data = IMDBData("train", TRAIN_PER_CLASS)
test_data = IMDBData("test", TEST_PER_CLASS)
print(f"读取耗时 {time.time() - t0:.1f}s")
print(f"训练集 {len(train_data)} 条，测试集 {len(test_data)} 条")
print(f"正样本占比：训练 {np.mean(train_data.labels):.2f}")
```

> 💡 **提示**：这里为什么要打乱？tar 包里的文件是“train/neg 全部在前、train/pos 在后”的顺序存的，如果直接按顺序取前 80% 当训练集，训练集里可能全是差评，模型就只能学会“永远猜差评”。打乱之后再切分才能保证训练集正负均衡。

> ✅ **检查点**：打印出的训练集条数是 25000（每类 12500），正样本占比 0.50。

任务四：构建词表

统计训练集词频，只保留出现 ≥5 次的词，再补上两个特殊符号：

```
counter = collections.Counter(w for doc in train_data.docs for w in doc)
words = [w for w, cnt in counter.most_common() if cnt >= MIN_FREQ]
vocab_list = ["<pad>", "<unk>"] + words      # 0 号是补齐符，1 号是未知词
word2id = {w: i for i, w in enumerate(vocab_list)}

print(f"原始词表 {len(counter)} 个，保留 {len(vocab_list)} 个（含 2 个特殊符号）")
```

- `<pad>`：补齐用，index 固定为 0，对应的词向量是全 0，LSTM 看不到它
- `<unk>`：词表里没有的词（生僻词、拼写错误）统一映射到它

> ✅ **检查点**：打印出词表大小，约 3.2 万（不同训练集规模会略有差异）。

任务五：加载 GloVe 生成词向量矩阵

```
table = np.random.normal(0, 0.1, (len(vocab_list), EMBED_DIM)).astype(np.float32)
table[0] = 0.0           # <pad> 用全零向量

hit = 0
with open(GLOVE_PATH, encoding="utf-8") as fh:
    for line in fh:
        parts = line.rstrip().split(" ")
        idx = word2id.get(parts[0])
        if idx is None:          # 这个词不在我们的词表里，跳过
            continue
        table[idx] = np.asarray(parts[1:], dtype=np.float32)
        hit += 1

print(f"词向量命中 {hit}/{len(vocab_list)}，矩阵形状 {table.shape}")
```

> 💡 **提示**：`table` 就是 `nn.Embedding` 的初始权重。命中的词用预训练向量，没命中的保持随机值——训练时会慢慢学到合适的表示。

> 🔴 **截图 3**

```
…   词向量命中 28562/32065，矩阵形状 (32065, 100)
```

任务六：搭数据流水线

用 `mindspore.dataset` 的两步把“词序列”变成“定长 id 张量”：

1. `Lookup`：按词表把每个词换成 id，查不到的统一成 `<unk>` 的 id
2. `PadEnd`：不足 `MAX_LEN` 的补 0，超过的截断

```
vocab = ds.text.Vocab.from_list(vocab_list)
lookup_op = ds.text.Lookup(vocab, unknown_token="<unk>")
pad_op = ds.transforms.PadEnd([MAX_LEN], pad_value=word2id["<pad>"])
cast_op = ds.transforms.TypeCast(ms.float32)


def make_dataset(data, shuffle):
    dataset = ds.GeneratorDataset(data, column_names=["text", "label"],
shuffle=shuffle)
    dataset = dataset.map(operations=[lookup_op, pad_op], input_columns=["text"])
    dataset = dataset.map(operations=[cast_op], input_columns=["label"])
    return dataset


ds_train = make_dataset(train_data, shuffle=True)
ds_test = make_dataset(test_data, shuffle=False)

# 从训练集里再切 20% 出来当验证集，用来监控训练过程
ds_train, ds_valid = ds_train.split([0.8, 0.2])
ds_train = ds_train.batch(BATCH_SIZE, drop_remainder=True)
ds_valid = ds_valid.batch(BATCH_SIZE, drop_remainder=False)
ds_test = ds_test.batch(BATCH_SIZE, drop_remainder=False)

print(f"训练 batch 数 {ds_train.get_dataset_size()}，验证
{ds_valid.get_dataset_size()}，测试 {ds_test.get_dataset_size()}")
```

> ⚠️ **注意**：验证集和测试集都用了 `drop_remainder=False`，保证最后不满一批的样本也参与评估；只有训练集用 `True`，这样每批形状固定，训练更稳定（也避免样本太少时出现 0 个 batch 的除零错误）。

> ✅ **检查点**：打印出的训练 batch 数是 625（25000 条的 80% = 20000，再除以 32 取整）。

任务七：定义 RNN 模型

```
class RNN(nn.Cell):
    """Embedding 查表 → 双向 LSTM 读序列 → 全连接打分。"""

    def __init__(self, vocab_size, embedding_table, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size, embedding_table.shape[1],
            embedding_table=ms.Tensor(embedding_table),
            padding_idx=0,
        )
        self.rnn = nn.LSTM(
            embedding_table.shape[1], hidden_size,
            num_layers=1, bidirectional=True, batch_first=True,
        )
        self.fc = nn.Dense(hidden_size * 2, 1, weight_init="HeUniform",
bias_init="Zero")
```

```
    def construct(self, x):
        emb = self.embedding(x)                 # (B, 200, 100)
        _, (h, _) = self.rnn(emb)               # h: (2, B, 64)
        out = ops.concat((h[-2, :, :], h[-1, :, :]), axis=1)   # (B, 128)
        return self.fc(out)                     # (B, 1)


net = RNN(len(vocab_list), table, HIDDEN_SIZE)
print(net)
```

三个要点：

- `padding_idx=0`：告诉 Embedding“0 号是补齐符”，它的向量固定为 0，不参与训练
- `h[-2]` 是正向最后状态、`h[-1]` 是反向最后状态，拼起来才是 128 维
- 输出 1 个值（logit），后面用 Sigmoid 压到 0~1 才是概率

截图 4

```
RNN(
  (embedding): Embedding(vocab_size=32065, embedding_size=100, use_one_hot=False, embedding_table=Parameter (name=embedding.embedding_table, s
  (rnn): LSTM(
    (rnn): _DynamicLSTMCPUGPU()
    (dropout_op): Dropout(p=0.0)
  )
  (fc): Dense(input_channels=128, output_channels=1, has_bias=True)
)
```

任务八：训练

```
loss_fn = nn.BCEWithLogitsLoss(reduction="mean")
optimizer = nn.Adam(net.trainable_params(), learning_rate=LR)


def forward_fn(inputs, targets):
    logits = net(inputs)
    return loss_fn(logits, targets.reshape(-1, 1))   # 标签补一维，和 logits 对齐


grad_fn = ms.value_and_grad(forward_fn, None, net.trainable_params())
sigmoid = ops.Sigmoid()


def evaluate(dataset):
    """在数据集上算准确率：Sigmoid 后 >0.5 判为好评。"""
    correct, total = 0, 0
    for inputs, targets in dataset.create_tuple_iterator():
        pred = sigmoid(net(inputs)).asnumpy().ravel() > 0.5
        truth = targets.asnumpy().ravel() > 0.5
        correct += int((pred == truth).sum())
        total += targets.shape[0]
    return correct / total


def train_one_epoch(epoch):
    net.set_train(True)
    total_loss, steps = 0.0, 0
```

```
    for inputs, targets in ds_train.create_tuple_iterator():
        loss, grads = grad_fn(inputs, targets)
        optimizer(grads)
        total_loss += float(loss)
        steps += 1
        if steps % 50 == 0:
            print(f"  epoch {epoch} step {steps}/{ds_train.get_dataset_size()} "
                  f"loss {total_loss / steps:.4f}", flush=True)
    return total_loss / steps


history = []
for epoch in range(1, EPOCHS + 1):
    t_start = time.time()
    avg_loss = train_one_epoch(epoch)
    net.set_train(False)
    valid_acc = evaluate(ds_valid)
    test_acc = evaluate(ds_test)
    history.append((epoch, avg_loss, valid_acc, test_acc))
    print(f"[epoch {epoch}] loss {avg_loss:.4f}  valid {valid_acc:.4f}  "
          f"test {test_acc:.4f}  用时 {time.time() - t_start:.0f}s", flush=True)
```

> ⚠️ **注意**：`grad_fn` 的第三个参数必须是 `net.trainable_params()`。如果误写成 `optimizer.parameters`（优化器内部还带着步数、动量等状态），在网络里会传进一堆非权重张量，结果是“loss 在动、参数在变，但准确率怎么都不涨”。

> ✅ **检查点**：每轮结束会打印 `[epoch n] loss ... valid ... test ...`，`test` 后面的数字就是这一轮在测试集上的准确率，应该逐轮上升。

> 🔴 **截图 5**，最后一轮

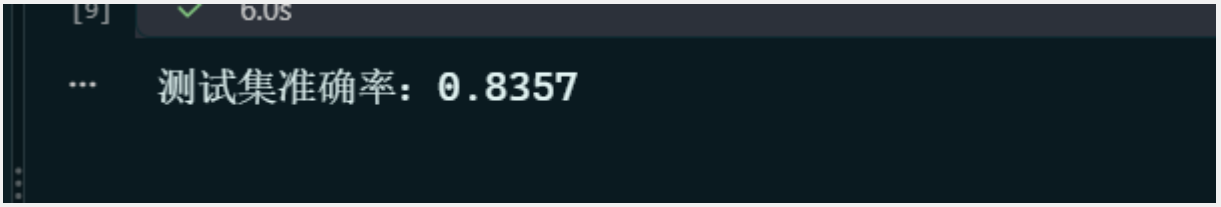
```
  epoch 3 step 550/625 loss 0.2291
  epoch 3 step 600/625 loss 0.2288
[epoch 3] loss 0.2291  valid 0.9522  test 0.8357  用时 124s
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

任务九：在测试集上评估

训练循环里每轮已经算过测试准确率，这里单独再跑一次，作为实验报告的最终结果：

```
net.set_train(False)
final_acc = evaluate(ds_test)
print(f"测试集准确率: {final_acc:.4f}")
```

> ✅ **检查点**：最终准确率应明显高于 0.50（随机猜），本实验约 0.83~0.86。

> 🔴 **截图 6**
>
> 
>

任务十：用模型给句子打分

```
def predict(text):
    tokens = text.replace("\n", " ").translate(PUNCT).lower().split()
    ids = [word2id.get(w, word2id["<unk>"]) for w in tokens][:MAX_LEN]
    if len(ids) < MAX_LEN:
        ids += [0] * (MAX_LEN - len(ids))
    logit = net(ms.Tensor([ids], ms.int32))
    score = float(sigmoid(logit).asnumpy().ravel()[0])
    return score, ("好评" if score > 0.5 else "差评")


samples = [
    "This movie is wonderful, the acting is great and the story is touching.",
    "What a waste of time, the plot is boring and the acting is terrible.",
    "I have seen better films, but it is not that bad.",
]
for s in samples:
    score, tag = predict(s)
    print(f"{tag}（{score:.3f}）  {s}")
```

参考输出（本机 2 轮训练后实测，你的数值会有出入）：

```
好评（0.962）  This movie is wonderful, the acting is great and the story is
touching.
差评（0.023）  What a waste of time, the plot is boring and the acting is
terrible.
差评（0.195）  I have seen better films, but it is not that bad.
```

第三条 `I have seen better films, but it is not that bad.`（"看过更好的，但也没那么差"）被判成差评，是**刻意留下的例子**：模型主要靠 `waste`、`boring`、`great` 这类强情感词做判断，对"双重否定 + 转折"这种需要真正的语义理解才能拿准。你可以把这条写进思考题，讨论"准确率 0.85 的模型在哪些句子上一定出错"。

> 💡 **提示**：模型只认识训练时见过的词。你写的句子里如果有大量生僻词，它们全变成 `<unk>`，判断就会不准——这也是词表覆盖率的重要之处。

截图 7

```
11]  ✓ 0.0s

好评（0.989）  This movie is wonderful, the acting is great and the story is touching.
差评（0.019）  What a waste of time, the plot is boring and the acting is terrible.
差评（0.500）  I have seen better films, but it is not that bad.
```

七、实验结果与思考题

7.1 参考结果

本实验在服务器（CPU 模式）上的实测：

轮数	训练 loss	验证集准确率	测试集准确率	本轮耗时
1	0.6158	0.7746	0.7352	120s
2	0.3770	0.9190	0.8405	122s
3	0.2328	0.9588	0.8417	118s

最终测试准确率 **0.8417**，3 轮总耗时约 360 秒（加上读数据约 370 秒）。

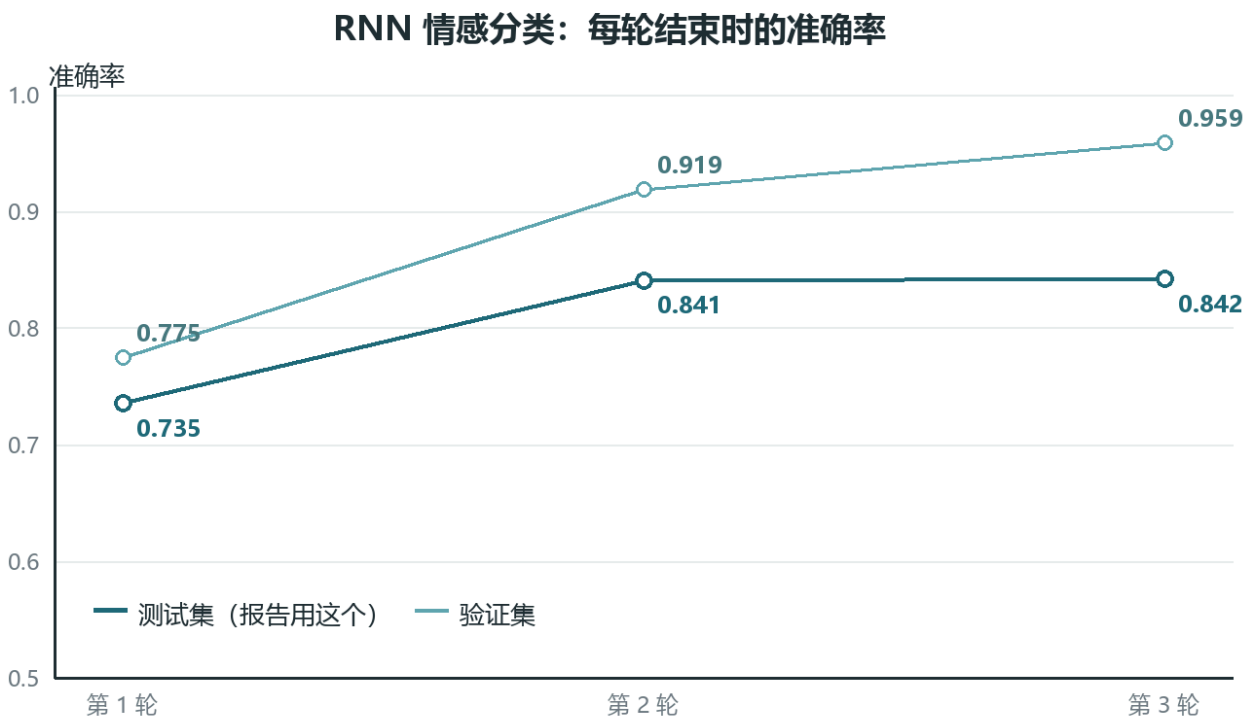


图 2　每轮结束时验证集与测试集准确率的变化（本机实测）

从曲线上能看出两件事：第 1 轮到第 2 轮涨得最快（模型先学会抓最容易区分的情感词），第 2 轮之后测试准确率基本走平（0.8405 → 0.8417），而验证集还在涨——这正是**过拟合**的开始。所以本实验把轮数定在 3，再往上加只是白烧 CPU。

> 💡 **提示**：验证集准确率通常比测试集高 5~12 个百分点（本机第 3 轮是 0.9588 对 0.8417），这不是代码写错了。验证集是从训练集里切出来的，和训练数据的来源、用词分布更像，模型自然"见过类似

> 的”；测试集是另一批影评，更严格。写报告时**用测试集准确率作为结论**。

7.2 思考题

1. 如果把 `MAX_LEN` 从 200 降到 50，准确率大概会怎么变？为什么？先把你的预测写下来，再跑一次验证。
2. 把词表的最小词频 `MIN_FREQ` 从 5 改成 1，词表会变大多少？准确率会变好吗？内存和时间有什么变化？
3. 双向 LSTM 比单向多了一半的计算量，它换来了什么？在什么任务上双向是不允许用的？（提示：想想“实时逐词输出”的场景。）
4. 为什么用 `BCEWithLogitsLoss` 而不是先用 Sigmoid 再算交叉熵？两者在数值上有什么区别？
5. 训练 loss 一直在降，但测试准确率某一轮之后不再上升，这说明什么？可以怎么缓解？

八、常见问题排查

报错 / 现象	原因	处理
`No module named 'tbe'` 或 `Environment_Error_Import_Python_Module_Failed`	没指定 CPU，MindSpore 去初始化昇腾芯片了	任务一的第一格必须执行 `ms.set_context(..., device_target="CPU")`
`FileNotFoundError: aclImdb_v1.tar.gz`	代码不在数据目录下运行	`cd` 到放数据文件的目录，或把 `DATA_DIR` 改成绝对路径
`Lookup` 报 `special_tokens and word_list contain duplicate word`	词表里已经有 `<pad>`、`<unk>`，又重复传了 `special_tokens`	用本指导书的写法：先把两个特殊符号放进 `vocab_list`，`Vocab.from_list` 不再传 `special_tokens`
`BCEWithLogitsLoss` 报 shape 不一致 `[32, 1]` vs `[32]`	标签少一维	`targets.reshape(-1, 1)`
准确率一直是 0.50	训练集里只有一个类别（数据没打乱就切片）	确认 `IMDBData` 里做了 shuffle
准确率一直是 1.00（假象）	同上，正负样本被彻底分开，模型在“猜常量”	同上，检查正样本占比是否 0.5
训练特别慢（每轮 5 分钟以上）	`TRAIN_PER_CLASS`、`MAX_LEN` 太大，或机器上有别人在跑	先把 `TRAIN_PER_CLASS` 降到 5000、`EPOCHS` 降到 3
`Cannot find the tbe operator implementation` 告警刷屏	昇腾环境变量没配	不影响 CPU 运行，忽略

九、附录 A：完整源码

把下面整段保存为 `imdb_rnn.py`，与 `aclImdb_v1.tar.gz`、`glove_imdb_100d.txt` 放同一目录，`python3 imdb_rnn.py` 即可一次跑完（约 6~8 分钟）。

```
import collections
import os
import re
import string
import tarfile
import time

import numpy as np
import mindspore as ms
import mindspore.dataset as ds
import mindspore.nn as nn
import mindspore.ops as ops

ms.set_context(mode=ms.PYNATIVE_MODE, device_target="CPU")

DATA_DIR = "."
TAR_PATH = os.path.join(DATA_DIR, "aclImdb_v1.tar.gz")
GLOVE_PATH = os.path.join(DATA_DIR, "glove_imdb_100d.txt")

MAX_LEN = 200
BATCH_SIZE = 32
EPOCHS = 3
LR = 1e-3
HIDDEN_SIZE = 64
EMBED_DIM = 100
MIN_FREQ = 5
TRAIN_PER_CLASS = 12500
TEST_PER_CLASS = 2000

PAT = re.compile(r"aclImdb/(train|test)/(pos|neg)/.*\.txt$")
PUNCT = str.maketrans("", "", string.punctuation)


class IMDBData:
    def __init__(self, split, per_class, seed=42):
        docs, labels = [], []
        counts = {1.0: 0, 0.0: 0}
        with tarfile.open(TAR_PATH) as tarf:
            for member in tarf:
                m = PAT.match(member.name)
                if not m or m.group(1) != split:
                    continue
                label = 1.0 if m.group(2) == "pos" else 0.0
                if counts[label] >= per_class:
                    continue
                raw = tarf.extractfile(member).read().decode("utf-8", "ignore")
                docs.append(raw.replace("\n", "
").translate(PUNCT).lower().split())
                labels.append(label)
```

```
                counts[label] += 1
                if counts[0.0] >= per_class and counts[1.0] >= per_class:
                    break
        order = np.random.default_rng(seed).permutation(len(docs))
        self.docs = [docs[i] for i in order]
        self.labels = [labels[i] for i in order]

    def __getitem__(self, idx):
        return self.docs[idx], self.labels[idx]

    def __len__(self):
        return len(self.docs)


class RNN(nn.Cell):
    def __init__(self, vocab_size, embedding_table, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size, embedding_table.shape[1],
            embedding_table=ms.Tensor(embedding_table), padding_idx=0,
        )
        self.rnn = nn.LSTM(embedding_table.shape[1], hidden_size,
                           num_layers=1, bidirectional=True, batch_first=True)
        self.fc = nn.Dense(hidden_size * 2, 1, weight_init="HeUniform",
bias_init="Zero")

    def construct(self, x):
        _, (h, _) = self.rnn(self.embedding(x))
        return self.fc(ops.concat((h[-2, :, :], h[-1, :, :]), axis=1))


def main():
    t0 = time.time()
    train_data = IMDBData("train", TRAIN_PER_CLASS)
    test_data = IMDBData("test", TEST_PER_CLASS)
    print(f"读取 {time.time() - t0:.1f}s  train={len(train_data)} test=
{len(test_data)}")

    counter = collections.Counter(w for doc in train_data.docs for w in doc)
    vocab_list = ["<pad>", "<unk>"] + [w for w, c in counter.most_common() if c >=
MIN_FREQ]
    word2id = {w: i for i, w in enumerate(vocab_list)}
    print(f"词表 {len(vocab_list)}")

    table = np.random.normal(0, 0.1, (len(vocab_list),
EMBED_DIM)).astype(np.float32)
    table[0] = 0.0
    hit = 0
    with open(GLOVE_PATH, encoding="utf-8") as fh:
        for line in fh:
            parts = line.rstrip().split(" ")
            idx = word2id.get(parts[0])
            if idx is None:
                continue
```

```
            table[idx] = np.asarray(parts[1:], dtype=np.float32)
            hit += 1
    print(f"词向量命中 {hit}/{len(vocab_list)}")

    vocab = ds.text.Vocab.from_list(vocab_list)
    lookup_op = ds.text.Lookup(vocab, unknown_token="<unk>")
    pad_op = ds.transforms.PadEnd([MAX_LEN], pad_value=0)
    cast_op = ds.transforms.TypeCast(ms.float32)

    def make_dataset(data, shuffle):
        dataset = ds.GeneratorDataset(data, column_names=["text", "label"],
shuffle=shuffle)
        dataset = dataset.map(operations=[lookup_op, pad_op], input_columns=
["text"])
        dataset = dataset.map(operations=[cast_op], input_columns=["label"])
        return dataset

    ds_train = make_dataset(train_data, shuffle=True)
    ds_test = make_dataset(test_data, shuffle=False)
    ds_train, ds_valid = ds_train.split([0.8, 0.2])
    ds_train = ds_train.batch(BATCH_SIZE, drop_remainder=True)
    ds_valid = ds_valid.batch(BATCH_SIZE, drop_remainder=False)
    ds_test = ds_test.batch(BATCH_SIZE, drop_remainder=False)

    net = RNN(len(vocab_list), table, HIDDEN_SIZE)
    loss_fn = nn.BCEWithLogitsLoss(reduction="mean")
    optimizer = nn.Adam(net.trainable_params(), learning_rate=LR)

    def forward_fn(inputs, targets):
        return loss_fn(net(inputs), targets.reshape(-1, 1))

    grad_fn = ms.value_and_grad(forward_fn, None, net.trainable_params())
    sigmoid = ops.Sigmoid()

    def evaluate(dataset):
        correct, total = 0, 0
        for inputs, targets in dataset.create_tuple_iterator():
            pred = sigmoid(net(inputs)).asnumpy().ravel() > 0.5
            truth = targets.asnumpy().ravel() > 0.5
            correct += int((pred == truth).sum())
            total += targets.shape[0]
        return correct / total

    for epoch in range(1, EPOCHS + 1):
        t_start = time.time()
        net.set_train(True)
        total_loss, steps = 0.0, 0
        for inputs, targets in ds_train.create_tuple_iterator():
            loss, grads = grad_fn(inputs, targets)
            optimizer(grads)
            total_loss += float(loss)
            steps += 1
            if steps % 50 == 0:
                print(f"  epoch {epoch} step {steps}/{ds_train.get_dataset_size()}
```

```
"
                      f"loss {total_loss / steps:.4f}", flush=True)
        net.set_train(False)
        valid_acc = evaluate(ds_valid)
        test_acc = evaluate(ds_test)
        print(f"[epoch {epoch}] loss {total_loss / steps:.4f}  valid
{valid_acc:.4f}  "
              f"test {test_acc:.4f}  {time.time() - t_start:.0f}s", flush=True)

    print(f"最终测试集准确率 {evaluate(ds_test):.4f}")


if __name__ == "__main__":
    main()
```

十、附录 B：耗时与调参记录

服务器实测（openEuler 22.03 aarch64，CPU 模式，机器空闲时）：

配置	每轮耗时	6 轮总耗时	测试准确率
全量 25000 条 / MAX_LEN 200 / 每轮 625 batch	约 120 秒	约 370 秒	0.8417
5000 条 / MAX_LEN 200 / 3 轮	约 15 秒	约 1 分钟	约 0.77
全量 25000 条 / MAX_LEN 400	约 190 秒	约 19 分钟	略高但得不偿失

调参建议：

- **想更快**：`TRAIN_PER_CLASS=5000`、`EPOCHS=3`，2 分钟内出结果，准确率约 0.77~0.80。
- **想更准**：保持全量、`EPOCHS` 加到 8~10，但 CPU 上一轮约 70 秒，总时间会到 12 分钟以上；再往上收益很小。
- **别动**：`EMBED_DIM` 必须和 GloVe 文件一致（100），改了会直接报矩阵维度不匹配。
- 机器上有别人在跑任务时速度会明显变慢（CPU 抢占），出现“比平时慢一倍”属正常。

参考资料

1. MindSpore 官方教程：RNN 实现情感分类
2. GloVe: Global Vectors for Word Representation（Pennington et al., 2014）
3. IMDB Movie Reviews Dataset（Maas et al., 2011）
4. Understanding LSTM Networks（Chris Olah, 2015）